

ITC MP

Correction DS3

Ce document contient des éléments de correction pour le sujet 2025 des mines. Cela reste assez succinct, donc n'hésitez pas à me poser des questions par mail si besoin.

1)

1)1)

```
def degreMax(d):  
    m = 0  
    for s, l in d.items():  
        if len(l) > m:  
            m = len(l)  
    return m
```

1)2)

```
def graphInv(d):  
    dinv = {v: [] for v, _ in d.items()}  
    for s1, l in d.items():  
        for s2 in l:  
            dinv[s2].append(s1)  
    return dinv
```

1)3)

```
def colorationValide(d, L):  
    for s1, l in d.items():  
        for s2 in l:  
            if L[s1] == L[s2]:  
                return False  
    return True
```

1)4)

$O(N+M)$

1)5)

```
SELECT nomfilm, duree FROM films JOIN locations  
ON films.codefilm = locations.codefilm ORDER BY duree DESC LIMIT 1
```

1)6)

```
SELECT F.codefilm, nomfilm, AVG(duree) as duree_moyenne FROM films as F JOIN  
locations as L  
ON F.codefilm = L.codefilm GROUP BY codefilm HAVING duree_moyenne < 2 ORDER BY  
duree_moyenne DESC
```

2)

2)1)

```
SELECT idC, val / taille as rentabilite FROM Conteneurs WHERE  
portDepC = "Marseille" AND portDestC = "Barcelone" AND dateDisp < 2025-01-01  
ORDER BY rentabilite DESC;
```

2)2)

Il n'y avait pas besoin de JOIN, tout était dans Conteneurs

```
SELECT idN, COUNT() as nbC FROM Conteneurs GROUP BY idN WHERE idN != 0;
```

2)3)

```
def profit(obj, S):
    total = 0
    for i in range(len(obj)):
        total += obj[i][1] * S[i]
    return total
```

2)4)

```
def contrainte(obj, S, b):
    capacite = 0
    for i in range(len(obj)):
        capacite += obj[i][0] * S[i]
    return capacite <= b
```

2)5)

```
1 0 1
1 1 0
```

2)6)

2^n feuilles, donc $O(n2^n)$. Attention, le n ne disparaît pas !

2)7)

```
1 1 0
```

2)8)

```
Lqi.append(obj[i][1] / obj[i][0])
...
Li[j] = Li[j-1]
...
Li[j] = i
```

2)9)

$O(n)$ ($O(n^2)$)

2)10)

```
while j < len(obj):
    if obj[Li[j]][0] <= b:
        S[Li[j]] = 1
        b = b - objx[Li[j]][0]
    j += 1
```

2)11)

La solution optimale est 0 1 1, le glouton n'est donc pas optimal. On pouvait commenter sur la différence de poids choisie, qui était assez faible.

2)12)

2)13)

$O(bn)$

2)14)

La complexité n'était pas modifiée.

2)15)

```
def est_feuille(a):  
    return a['g'] == {}
```

2)16)

```
def possible(obj, Sk, b):  
    n = len(obj)  
    k = len(Sk)  
    Smin = Sk + (n-k) * [0]  
    Smax = Sk + (n-k) * [1]  
    return contrainte(obj, Smin, b) and profit(obj, Smax) > Pmin
```

2)17)

```
else:  
    if possible(obj, arbre['g']['S'], b):  
        KPpse(arbre['g'], obj, b)  
    if possible(obj, arbre['d']['S'], b):  
        KPpse(arbre['d'], obj, b)
```

2)18)

```
for i in range(len(T)):  
    T[i] *= rho
```

2)19)

```
kmax = 0  
Pmax = 0  
for i in range(len(S)):  
    P = profit(obj, S[i])  
    if P > Pmax:  
        kmax = i  
        Pmax = P
```

2)20)

```
for i in range(len(T)):  
    if S[kmax][i] == 1:  
        T[i] += 1/(1+PbestOfAll-Pmax)  
        T[i] = max(T[i], Tmin)  
        T[i] = min(T[i], Tmax)
```

2)21)

```
o0 = randint(0, n-1)  
...  
S[k][o0] = 1  
b2 -= obj[o0][0]
```

2)22)

```
prob = {}  
...  
oi = candidats[i]  
prob[oi] = (T[oi]**alpha)*((b * obj[oi][1] / obj[oi][0])**beta)  
...
```

```
oi = candidats[i]
prob[oi] /= s
```

2)23)

```
prob = construitProb(obj, candidats, b, T)
c = choixCandidats(candidats, prob)
S[k][candidats[c]] = 1
b2 -= obj[candidats[c]][0]
candidats_2 = []
for cc in candidats:
    if cc != c and b2 > obj[cc][0]:
        candidats_2.append(cc)
candidats = candidats_2
```

2)24)

Il n'y a pas une seule bonne réponse à ce genre de questions, mais il fallait remarquer que l'algorithme de progdyn allait vite et était optimal. Les algorithmes d'approximation peuvent aller encore plus vite si on accepte un résultat légèrement sous-optimal (en particulier la colonie de fourmis, qui a l'avantage d'être adaptable)